Abstraction Regret: Memory-Layout and Execution-Paradigm Constraints in GPU DSLs for Irregular Automata Processing

Alessandro Potenza
gpufsm project
ap.alessandro.potenza@gmail.com

***Abstract*—GPU domain-specific languages (DSLs) such as OpenAI Triton promise near-CUDA performance at far lower programming effort, a promise borne out on *regular* tensor algebra. We ask whether it holds for an *irregular* workload—non-deterministic finite automata (NFA) processing—and find that it does not, in an instructive way. We introduce *abstraction regret*: the performance a DSL forecloses because its abstraction cannot express the memory layout *or the control flow* a workload needs. We make it measurable with a two-parameter cost model and a controlled, constant-algorithm ablation across four memory axes (byte→bit working set, global→shared CSR, sync→async transfer, single→multi-stream) and three DSLs (CUDA, Triton, NVIDIA Warp), plus a feasibility study of Triton's Gluon frontend. Two findings stand out. First, on a faithful full-scan NFA kernel the bottleneck is *compute*, not memory: staging the transition table into shared memory (zero global traffic) leaves throughput unchanged, and throughput scales as $1/n^2$ with state count—so memory-layout techniques cannot help until the algorithm is made work-efficient. We then build a work-efficient active-set kernel that is $250\times$–$10^4\times$ faster and reaches 15–142 Gbps. Second, the Triton↔CUDA gap is a per-DSL constant ($\approx 15.7\times$ on this kernel) that no traffic/compute term explains, while Warp—an equally high-level *Python* DSL—matches or beats hand-written CUDA ($0.62\times$). Abstraction regret is therefore set by the execution *paradigm* (tile/SPMD vs thread-SIMT), not by how high-level the DSL looks; and expressibility does not buy efficiency: Triton *can* express the work-efficient kernel yet still pays $\approx 9\times$ there.**

I. Introduction

Irregular workloads—graph traversal, sparse algebra, automata—are dominated by data layout and data-dependent control flow rather than arithmetic. GPU DSLs raise the abstraction level by hiding exactly those concerns (thread indexing, memory placement, control flow), which is why they excel on dense tensor kernels. This paper measures what that hiding costs on NFA processing, a canonical irregular workload (deep-packet inspection, regular-expression matching, bioinformatics).

Contributions. (A) *Abstraction regret, operationalized*: a named framing, a predictive two-parameter cost model (§III), and a constant-algorithm factorial ablation (§VI) that attribute the DSL gap to *expressible memory layout and control flow*, distinct from generic performance portability and from auto-tuning. **(B)** *A work-efficient portable NFA engine*: an active-set/worklist bit-packed kernel (§IV) that removes the $O(n^2)$ compute wall and reaches 15–142 Gbps. **(C)** *A reproducible artifact*: one registry-based framework, a CPU reference oracle, median+CI95 sweeps, and figures regenerated only from versioned CSVs.

II. Background

NFA simulation maintains an active-state set and, per input symbol, applies the transition relation and an epsilon-closure. GPU engines since iNFAnt [1] store transitions in symbol-indexed / CSR form and represent the active set as a bit-vector. The modern frontier—AsyncAP [2], ngAP [3] (non-blocking + memoization + privatization), HybridSA [4] (bit-parallel), BitGen [5] (Parabix bitstreams)—wins by reorganizing or reducing irregular memory traffic, yet each frames its contribution as a new algorithm and bundles the memory effects in. Hyperscan [6] is the CPU baseline; ANMLZoo [7] and AutomataZoo are the standard suites. No prior GPU-automata work uses Triton, isolates the memory axes at constant algorithm, or frames the DSL gap as an expressibility cost.

III. Abstraction regret and the cost model

We model per-symbol time as a roofline-style sum:

$$t_{\text{sym}} = \frac{\text{traffic}_{\text{bytes/sym}}}{\text{bandwidth}} + n^2 \cdot c_2, \tag{1}$$

where n is the state count. The compute term is *quadratic* because the faithful kernel performs an $O(n)$ transition scan and an $O(n^2)$ epsilon-closure (n convergence passes $\times\, n$ states) per symbol. The two constants are fitted per backend from measured throughput; the ratio of c_2 between two DSLs, at constant algorithm, *is* the abstraction regret. The per-backend fit tracks the measurements closely (Fig. 1).

IV. Implementation

A single registry maps a backend/technique to an executor. The NFA is stored once in CSR and consumed

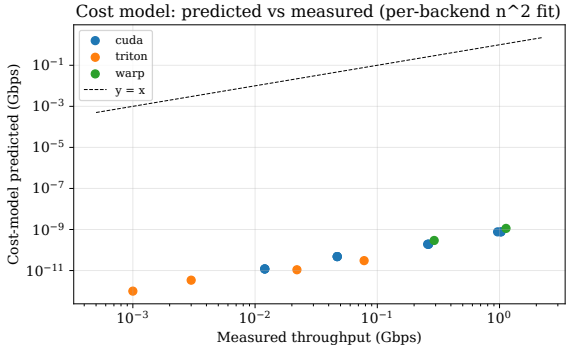

Fig. 1. Cost-model predicted vs measured throughput (per-backend n^2 fit); points near $y{=}x$ indicate the two-parameter model captures the regime.

identically by every backend, so comparisons are apples-to-apples. Correctness is gated against a CPU reference oracle (latch-first-match) and its bit-packed executable spec. **CUDA**: `dense` (int8/state), `bitpacked` (register-resident 64-bit words), `multistream` (one thread/string), `multistream_shared` (CSR staged in shared memory—a layout Triton cannot express), `multistream_async` (pinned + streamed overlap), and `worklist` (active-bit iteration via `__ffsll` + frontier eps-closure; O(active), no $O(n^2)$), and `worklist_global` (global working set, dynamic word count, no 512-state cap — scales to ANMLZoo-sized automata; register residency costs ∼4–5× vs the capped kernel). **Triton**: `dense`, `bitpacked`, `multistream`, `worklist` ($\leq$ 64 states, via `libdevice.ffs` + a data-dependent `while` loop). **Warp**: `multistream` (thread-SIMT Python). **Gluon** (Triton's experimental low-level frontend): `gl.load` always returns a layout-typed tensor (no scalar load), so the data-dependent CSR inner loop is inexpressible.

V. Methodology

Throughput is measured on a fixed batch (2048 × 256 B) as total input bits per batch kernel time, reported as median + percentile-bootstrap 95% CI over 9 runs (GPU timings are non-Gaussian [8]), kernel and transfer time separated. Every configuration is verified bit-identical to the oracle on examples and randomized fuzz/stress NFAs (up to 500 states). Data and library versions are captured per CSV row. Hardware: NVIDIA RTX 4070 (sm_89, 46 SMs, 12 GB); software: CUDA toolkit 13.x, Triton 3.5.1, Warp 1.14, PyTorch 2.9 (cu128). Each datum is the median of 9 batch-kernel times; CSR transitions are read-only and shared across the batch. We additionally validate on *real* ANMLZoo automata (Levenshtein 2787 states; Hamming 11349 states, 2.1M transitions; Brill 42661 states, 4.4M transitions; loaded from pinned public ANML with correct `all-input`/`start-of-data` semantics): `worklist_global` matches the reference oracle bit-for-bit on every input, confirming correctness at production scale (tens of thousands of states).

TABLE I
Throughput (Gbps) by technique and NFA size (batched, RTX 4070). Dashes: > 64 states unsupported by the single-word Triton/Warp kernels.

states	CUDA worklist	CUDA full-scan	Triton full-scan	Triton worklis
32	142.2	0.513	0.071	24.5
64	142.2	0.131	0.021	25.1
128	45.5	0.023	0.003	-
256	33.9	0.006	0.001	-
500	15.3	0.0015	0.0002	-

TABLE II
Nsight Compute counters (single launch). Full-scan: SM throughput ≫ DRAM ⇒ compute-bound; shared-CSR is identical but for L2 hit rate.

kernel	SM%	DRAM%	L2 hit%	occ%
`multistream` (full-scan)	19.44	0.01	79.2	16.7
`multistream_shared`	19.60	0.01	93.0	16.7
`worklist` (16384 str)	5.43	6.30	95.6	23.4

VI. Results

Table I reports throughput; Table II the Nsight counters.

The faithful kernel is compute-bound; memory layout is irrelevant there. `multistream`, `multistream_shared` (modeled CSR traffic = 0), and `multistream_async` tie to within the bootstrap CI at every size (Fig. 4), and throughput scales as $1/n^2$ (Fig. 2). Nsight Compute counters confirm this at the hardware level: the full-scan kernel sustains ≈19% of peak SM throughput but only **0.01%** of peak DRAM throughput, and `multistream_shared` shows identical SM%, DRAM% and occupancy—only the L2 hit rate rises (79→93%), without moving runtime. The memory axes cannot help until the algorithm is work-efficient.

The work-efficient kernel unlocks the regime. The `worklist` kernel is 250×–10^4× faster than full-scan (the speedup grows with n; Fig. 3) and reaches 15–142 Gbps, moving the workload toward memory-bound where the memory techniques become load-bearing.

Abstraction regret is the execution paradigm. On the same full-scan kernel the fitted per-DSL compute cost vs CUDA is Triton 15.7×, CUDA 1.0×, Warp 0.62× (Fig. 5). Two equally high-level Python DSLs sit at opposite extremes: Triton's tile/SPMD paradigm strains to express per-state data-dependent control flow, Gluon cannot express it at all, while Warp's thread-SIMT model expresses it naturally and its codegen beats hand-written CUDA.

Expressibility ≠ efficiency. Triton *can* express the work-efficient worklist (unlike Gluon), yet still pays ≈ 9× vs CUDA there (CUDA 142 Gbps vs Triton 25 Gbps at $\leq$ 64 states), versus 15.7× on the full scan. Even expressing the right algorithm, the tile/SPMD model imposes a large constant penalty on scalar, data-dependent automata work.

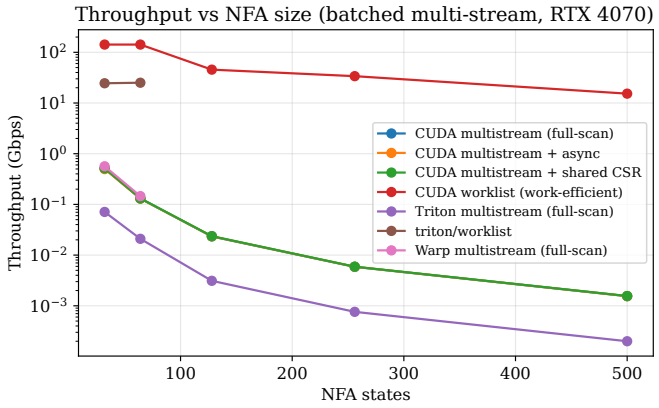

Fig. 2. Throughput vs NFA size (log scale). Worklist dominates; full-scan techniques collapse as $1/n^2$.

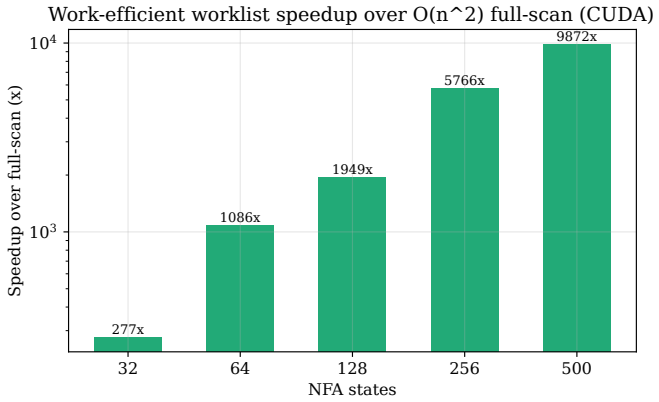

Fig. 3. Work-efficient worklist speedup over $O(n^2)$ full-scan (CUDA), growing with state count.

VII. Related Work

Performance portability [9] measures efficiency across a *hardware set*; abstraction regret measures efficiency across an *abstraction axis at fixed hardware*, attributed to expressibility. The Halide [10] lineage established that abstraction constrains the schedule space; we show the analogous constraint on control flow + memory layout for irregular automata. SpMV format selection and Gunrock [11] establish that layout dominates irregular GPU performance; we add the DSL-expressibility axis. We rebut the autotuning counter-thesis: our gap is layout/control-flow expressibility, not tuning—Triton cannot express the shared-CSR layout at any tuning. SOTA automata engines (ngAP, HybridSA, BitGen) are baselines our worklist engine approaches; our contribution is the framing + cost model + the multi-DSL expressibility study.

VIII. Limitations and Future Work

Single GPU architecture so far; a second architecture is needed for generality. The worklist is one thread/string and, at small batches, under-utilizes the GPU (Nsight: 17% occupancy, 2 blocks); a cooperative warp/block-parallel version is the path to SOTA absolute throughput when streams are few.

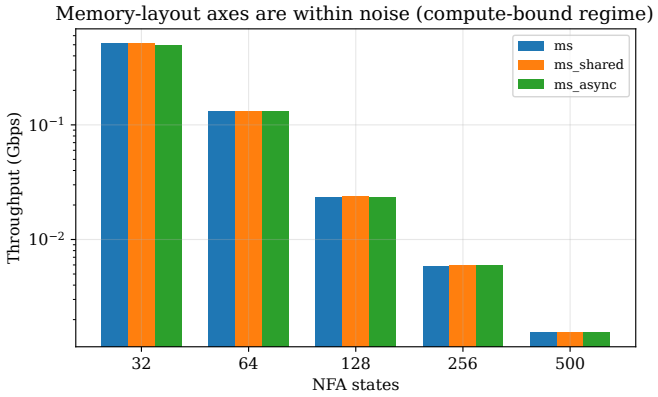

Fig. 4. `multistream` vs `_shared` vs `_async`: within noise at every size—compute-bound regime, memory layout irrelevant.

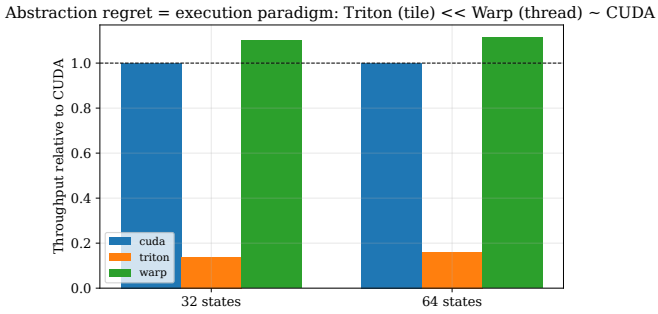

Fig. 5. Throughput relative to CUDA: Triton (tile) $\ll$ Warp (thread) $\approx$ CUDA. Regret is the execution paradigm, not abstraction height.

IX. Conclusion

For irregular automata the GPU-DSL promise breaks along a measurable axis we call abstraction regret, governed by expressibility—of memory layout and, more sharply here, of data-dependent control flow—not by abstraction height. The memory-organization thesis holds, but only once the algorithm is work-efficient enough to be memory-bound, which our worklist engine achieves.

References

[1] N. Cascarano *et al.*, "iNFAnt: NFA pattern matching on GPGPU devices," *ACM SIGCOMM CCR*, 2010.

[2] H. Liu, S. Pai, A. Jog, "Asynchronous Automata Processing on GPUs," *POMACS/SIGMETRICS*, 2023.

[3] T. Ge, T. Zhang, H. Liu, "ngAP: Non-blocking Large-scale Automata Processing on GPUs," *ASPLOS*, 2024.

[4] A. Le Glaunec, L. Kong, K. Mamouras, "HybridSA: GPU Acceleration of Multi-pattern Regex Matching using Bit Parallelism," *OOPSLA*, 2024.

[5] T. Ge, X. Chu, H. Liu, "Interleaved Bitstream Execution for Multi-Pattern Regex Matching on GPUs," *MICRO*, 2025.

[6] X. Wang *et al.*, "Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs," *USENIX NSDI*, 2019.

[7] J. Wadden *et al.*, "ANMLZoo: a benchmark suite for exploring bottlenecks in automata processing," *IISWC*, 2016.

[8] T. Hoefler, R. Belli, "Scientific Benchmarking of Parallel Computing Systems," *SC*, 2015.

[9] S. J. Pennycook, J. D. Sewall, V. W. Lee, "A Metric for Performance Portability," *PMBS@SC / FGCS*, 2016/2019.

[10] J. Ragan-Kelley *et al.*, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation," *PLDI*, 2013.

[11] Y. Wang *et al.*, "Gunrock: A High-Performance Graph Processing Library on the GPU," *PPoPP*, 2016.